PocketSmart AI: Your Smart Budget & Recommendation Assistant

Project Description:

Managing a budget across different needs such as home decor, event planning or jewelry shopping is overwhelming because of the many products, platforms and price ranges. PocketSmart AI is a Flask web application that uses Google Gemini to turn a user's budget, preferences and optional outfit image into a clear allocation plan with recommendations and platform search links.

After registering and logging in, a user chooses the Home Interior, Party or Jewelry planner, enters a budget and details, and receives a JSON-based plan that is shown as a budget summary (budget, planned, remaining), an allocation list, recommendation cards and tips. Every plan is saved in a SQLite database so the user can open it again from the dashboard. Secrets such as the Gemini API key are kept in a .env file, passwords are hashed and uploaded images are deleted after each request.

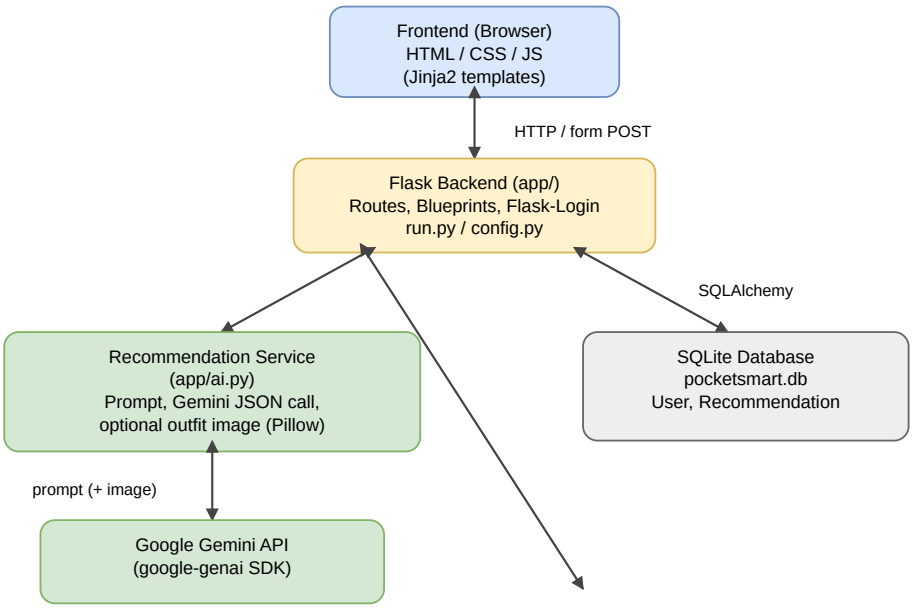
Scenarios

Scenario 1: Home Interior Planning with Smart Budget Allocation. A user opens the Home planner, enters a budget of Rs 50,000, the rooms (living room, bedroom), a Modern style and requirements such as a sofa for five people, two ceiling fans and six LED lights. Gemini funds the essentials first, then the TV unit, curtains and coffee table, keeps a small buffer and returns recommendations with Search on IKEA / Amazon / Flipkart links.

Scenario 2: Party Planning within a Budget. In the Party planner the user enters a budget, an event type such as a birthday, the number of guests, the city and preferences such as vegetarian food. The AI splits the budget across catering, venue, decoration, entertainment and contingency and suggests options that link to Zomato, Swiggy and OYO searches.

Scenario 3: Jewelry for an Occasion. In the Jewelry planner the user enters a budget, occasion, style and metal preference and may upload an outfit image (PNG, JPG, JPEG or WEBP up to 5 MB). Gemini uses the visible style and colour cues to suggest matching pieces, and the temporary image is deleted after the request.

Technical Architecture:



Description: PocketSmart AI is a modular three-layer Flask application. The browser layer is built from Jinja2 templates with HTML, CSS and a small JavaScript file. The Flask layer (app/) uses blueprints for authentication (auth.py) and the planners (main.py), Flask-Login for sessions and Werkzeug for password hashing. The recommendation service (ai.py) builds the prompt, calls Gemini through the google-genai SDK in JSON mode and can attach an outfit image opened with Pillow. Flask-SQLAlchemy stores users and saved plans in SQLite (pocketsmart.db).

Component-Wise Architecture

Component	Description
Frontend UI	Jinja2 templates (base, index, login, register, dashboard, home, party, jewelry, _result, history_detail) with style.css and app.js; forms collect budgets, preferences and images and the result partial shows the plan.
Flask Backend (app/)	create_app() loads Config, initialises SQLAlchemy and Flask-Login, registers auth_bp (/register, /login, /logout) and main_bp (/, /dashboard, /planner/<type>, /history/<id>, /api/health).
Recommendation Service (ai.py)	generate_recommendation() builds the prompt, calls the configured Gemini model (temperature 0.35, up to 5000 output tokens, JSON response) and parses the plan.
Platform Links	platform_url() turns a platform name and search phrase into a search URL for Amazon, Flipkart, IKEA, Swiggy, Zomato, OYO, Myntra or a general web search.
Data Store	SQLite tables user and recommendation (planner type, budget, input summary, result JSON, timestamp).

Pre-requisites:

- Python 3.11 or newer: https://www.python.org/
- A Google AI Studio account to create a Gemini API key: https://aistudio.google.com/
- Flask framework basics: https://flask.palletsprojects.com/
- Flask-SQLAlchemy and Flask-Login basics
- HTML, CSS, JavaScript and Jinja2 templating basics
- Git and a GitHub account (repository: github.com/santhoshcreater/pocket-smart-AI), a code editor such as Visual Studio Code and a terminal

Project Workflow:

1. Gemini AI Setup & Initialization	• Activity 1.1: Create a Gemini API key in Google AI Studio. • Activity 1.2: Store GEMINI_API_KEY, GEMINI_MODEL and SECRET_KEY in a .env file. • Activity 1.3: Validate the connection through the google-genai client.
2. Core Functionality Development	• Activity 2.1: Create the project structure, config.py and the create_app() factory. • Activity 2.2: Build the User and Recommendation models. • Activity 2.3: Add register, login and logout routes. • Activity 2.4: Add the planner route for Home, Party and Jewelry.
3. Backend Implementation	• Activity 3.1: Write the Gemini prompt and JSON parsing in ai.py. • Activity 3.2: Add input validation, image upload handling and platform search links. • Activity 3.3: Save each plan and add the dashboard, /history/<id> and /api/health routes.
4. UI Development	• Activity 4.1: Build the base layout, landing page, login and register pages. • Activity 4.2: Build the planner forms, result partial, dashboard and saved-plan page.
5. Testing & Optimization	• Activity 5.1: Test the three planners with real-world budgets. • Activity 5.2: Check that planned spend stays within budget and tune the prompt. • Activity 5.3: Test validation, uploads and load behaviour.

Milestone 1: Gemini AI Initialization

In this foundational stage the AI access for PocketSmart AI is set up. The application relies on Google Gemini to interpret the budget and preferences and to read the optional outfit image. Access is created once in Google AI Studio and then read by the application from environment variables, so no key ever appears in source code.

Activity 1.1: Create a Gemini API key

- Sign in to Google AI Studio with a Google account and accept the terms.
- Open the API keys page and choose Create API key.
- Copy the key and keep it private.

Activity 1.2: Configure the environment file

- Copy .env.example to .env and paste the key as GEMINI_API_KEY.
- GEMINI_MODEL selects the Gemini model and SECRET_KEY secures Flask sessions.
- .gitignore excludes .env so the key is never committed.

.env.example

```
# Copy this file to .env and fill in your Gemini API key.
GEMINI_API_KEY=your_gemini_api_key_here

# Current Gemini model. Change this if Google changes availability.
GEMINI_MODEL=gemini-3.8-flash

# Flask settings
SECRET_KEY=replace_this_with_a_long_random_secret
DATABASE_URL=sqlite:///pocketsmart.db
FLASK_DEBUG=1
```

Screenshot: .env.example (GEMINI_API_KEY, GEMINI_MODEL, SECRET_KEY, DATABASE_URL, FLASK_DEBUG)

Activity 1.3: Validate Gemini connectivity

- ai.py creates the client from the key in Config and stops with a clear message if the key is missing.
- The request sends the prompt (and the image for jewelry) to the model and expects JSON back.

app/ai.py

```python
def _client():
    api_key = current_app.config["GEMINI_API_KEY"]
    if not api_key:
        raise RuntimeError(
            "GEMINI_API_KEY is missing. Create a .env file and add your Google Gemini API key."
        )
    return genai.Client(api_key=api_key)
...

    if image_path:
        image = Image.open(image_path)
        contents.append(image)

    response = client.models.generate_content(
        model=model,
        contents=contents,
        config=types.GenerateContentConfig(
            temperature=0.35,
            max_output_tokens=5000,
            response_mime_type="application/json",
        ),
    )

    raw = response.text.strip()
    try:
        return json.loads(raw)
    except json.JSONDecodeError as exc:
        raise RuntimeError(f"Gemini returned invalid JSON: {exc}") from exc
```

Screenshot: Gemini client creation and generate_content call in app/ai.py

Milestone 2: Core Functionalities Development

This milestone builds the core of PocketSmart AI with Flask: configuration, the application factory, the database models and the login system. The Gemini logic is kept in one module (ai.py) so every planner can reuse it.

File Explorer Structure

Screenshot: project folder structure in the editor

Activity 2.1: Application factory and configuration

- config.py reads SECRET_KEY, DATABASE_URL (SQLite by default), GEMINI_API_KEY, GEMINI_MODEL and the upload rules (5 MB, PNG/JPG/JPEG/WEBP).
- create_app() loads the config, initialises SQLAlchemy and Flask-Login, registers both blueprints and runs db.create_all().

app/__init__.py

```python
login_manager.login_view = "auth.login"
login_manager.login_message_category = "warning"

def create_app():
    app = Flask(__name__)
    app.config.from_object(Config)

    os.makedirs(app.config["UPLOAD_FOLDER"], exist_ok=True)

    db.init_app(app)
    login_manager.init_app(app)

    from app.models import User, Recommendation
    from app.auth import auth_bp
    from app.main import main_bp

    app.register_blueprint(auth_bp)
    app.register_blueprint(main_bp)

    @login_manager.user_loader
    def load_user(user_id):
        return db.session.get(User, int(user_id))

    with app.app_context():
        db.create_all()

    return app
```

Screenshot: create_app() in app/__init__.py

Activity 2.2: Database models

- User stores name, unique email, password hash and creation time.
- Recommendation stores the planner type, budget, input summary and result JSON for each plan.

app/models.py

```python
class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(120), nullable=False)
    email = db.Column(db.String(255), unique=True, nullable=False, index=True)
    password_hash = db.Column(db.String(255), nullable=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

    recommendations = db.relationship(
        "Recommendation",
        backref="user",
        lazy=True,
        cascade="all, delete-orphan"
    )

    def set_password(self, password):
        self.password_hash = generate_password_hash(password)

    def check_password(self, password):
        return check_password_hash(self.password_hash, password)

class Recommendation(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey("user.id"), nullable=False)
    planner_type = db.Column(db.String(30), nullable=False)
    budget = db.Column(db.Float, nullable=False)
    input_summary = db.Column(db.Text, nullable=False)
    result_json = db.Column(db.Text, nullable=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
```

Screenshot: User and Recommendation models in app/models.py

Activity 2.3: Register, login and logout routes

- /register validates the fields, requires 8+ characters, rejects duplicate emails and saves a hashed password.
- /login checks the password and starts a Flask-Login session; /logout ends it.

app/auth.py

```python

@auth_bp.route("/login", methods=["GET", "POST"])
def login():
    if current_user.is_authenticated:
        return redirect(url_for("main.dashboard"))

    if request.method == "POST":
        email = request.form.get("email", "").strip().lower()
        password = request.form.get("password", "")

        user = User.query.filter_by(email=email).first()
        if not user or not user.check_password(password):
            flash("Invalid email or password.", "danger")
            return render_template("login.html")

        login_user(user, remember=False)
        return redirect(url_for("main.dashboard"))

    return render_template("login.html")

@auth_bp.route("/logout")
def logout():
    logout_user()
    return redirect(url_for("auth.login"))
```

Screenshot: login and logout routes in app/auth.py

Milestone 3: Backend Implementation

This milestone connects the forms to Gemini. One planner route serves all three planners, validates the input, calls generate_recommendation(), adds search links and stores the result.

Activity 3.1: Routes

Route	Purpose
/ and /dashboard	Landing page for visitors; dashboard with the planner cards and the latest 10 plans for logged-in users.
/planner/home, /planner/party, /planner/jewelry	GET shows the form; POST validates the input, calls Gemini, adds search links and saves the plan.
/history/<id>	Shows a saved plan; the query filters by record id and user id.
/register, /login, /logout	User registration, login and logout.
/api/health	Returns {"status": "ok"} for a quick health check.

Activity 3.2: Prompt and JSON plan (ai.py)

- The prompt fixes the currency (INR), the budget limit and the exact JSON structure: title, summary, budget, allocated_total, remaining_budget, allocations, recommendations, tips and disclaimer.
- Rules require allocated_total to equal the sum of the allocations, forbid invented product URLs and tell the model not to identify people in outfit images.

app/ai.py

```
Do not pretend that a product is live/in-stock or that a price is current unless the
application has supplied verified live commerce data. Product/platform links generated
by this application are search links, not proof of availability.

Return ONLY valid JSON matching the requested schema.

Important:
- Currency is INR.
- Keep total planned spend <= the user's budget.
- Use sensible Indian-market price estimates when exact live prices are unavailable.
- Clearly distinguish estimated prices from verified prices.
- Do not invent product URLs.
- For platform links, return a search URL that is generated from the product/platform name.
- Prefer realistic quantities and explain trade-offs.
- Never recommend spending more than the budget.
'''

def generate_recommendation(planner_type, payload, image_path=None):
```

Screenshot: prompt instructions in app/ai.py

Activity 3.3: Search links and saving history (main.py)

app/main.py

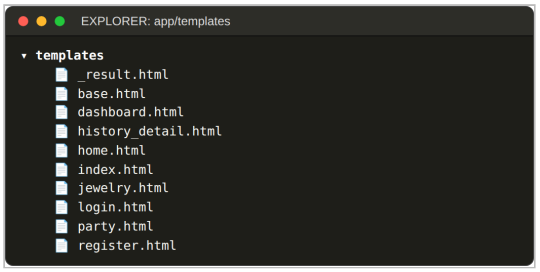
```python
    "home": "Home Interior",
    "party": "Party Planning",
    "jewelry": "Jewelry",
}

PLATFORMS = {
    "Amazon": "https://www.amazon.in/s?k={q}",
    "Flipkart": "https://www.flipkart.com/search?q={q}",
    "IKEA": "https://www.ikea.com/in/en/search/?q={q}",
    "Swiggy": "https://www.google.com/search?q=site%3Aswiggy.com+{q}",
    "Zomato": "https://www.google.com/search?q=site%3Azomato.com+{q}",
    "OYO": "https://www.google.com/search?q=site%3Aoyorooms.com+{q}",
    "Myntra": "https://www.myntra.com/{q}",
    "Other": "https://www.google.com/search?q={q}",
}

def platform_url(platform, query):
    template = PLATFORMS.get(platform, PLATFORMS["Other"])
    return template.format(q=quote_plus(query))

def allowed_image(filename):
...
            result = generate_recommendation(planner_type, payload, image_path)

            # Add clickable search links generated by the app.
            for item in result.get("recommendations", []):
                item["shopping_link"] = platform_url(
                    item.get("platform", "Other"),
                    item.get("search_query") or item.get("name", "")
                )

            record = Recommendation(
                user_id=current_user.id,
                planner_type=planner_type,
                budget=budget,
                input_summary=json.dumps(payload),
                result_json=json.dumps(result),
            )
            db.session.add(record)
            db.session.commit()

            if image_path and os.path.exists(image_path):
                os.remove(image_path)
```

Screenshot: PLATFORMS / platform_url() and saving the Recommendation in app/main.py

Milestone 4: UI Development

This milestone builds a lightweight, responsive interface with HTML, CSS and Jinja2 templates. base.html provides the navigation bar and footer for every page, and the planner pages share the _result.html partial for showing plans.

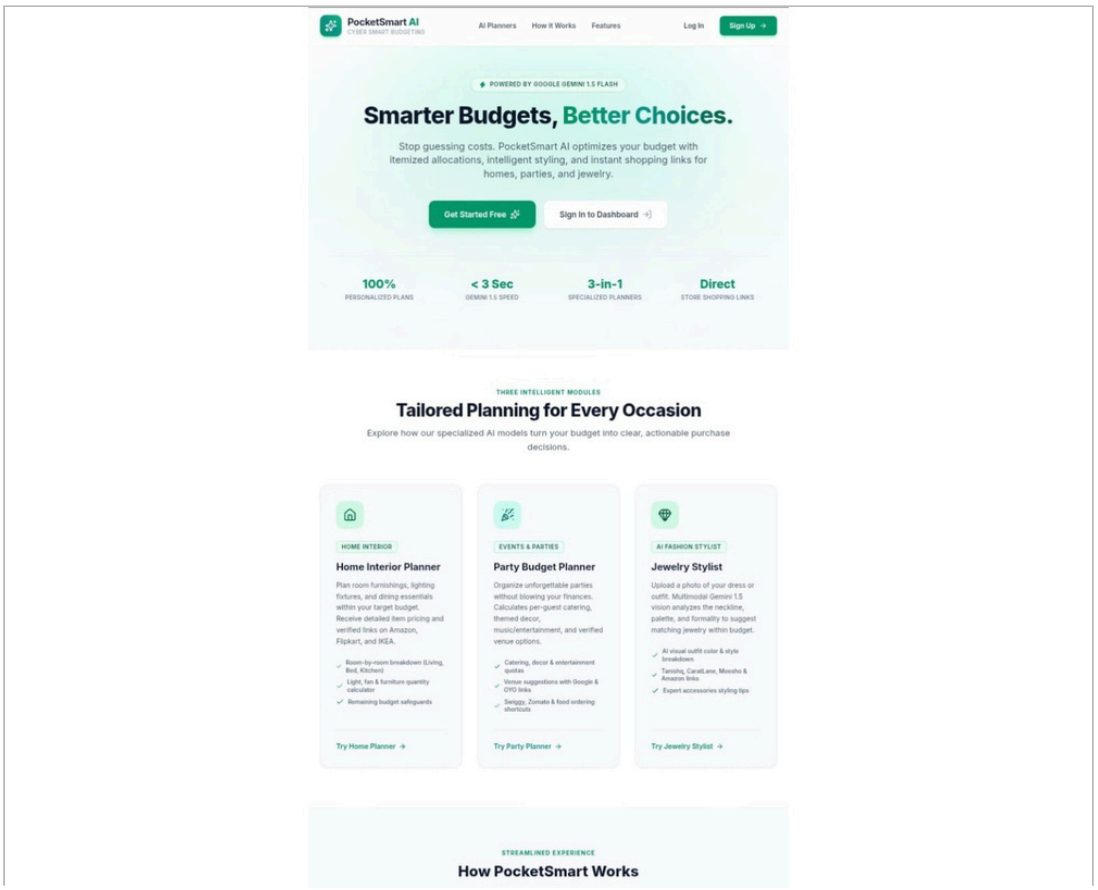
Templates Directory Structure



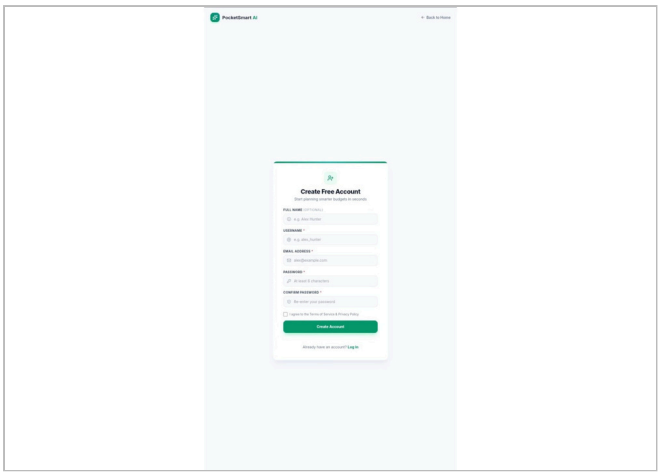
Screenshot: templates folder

Activity 4.1: Base layout, landing, register and login pages

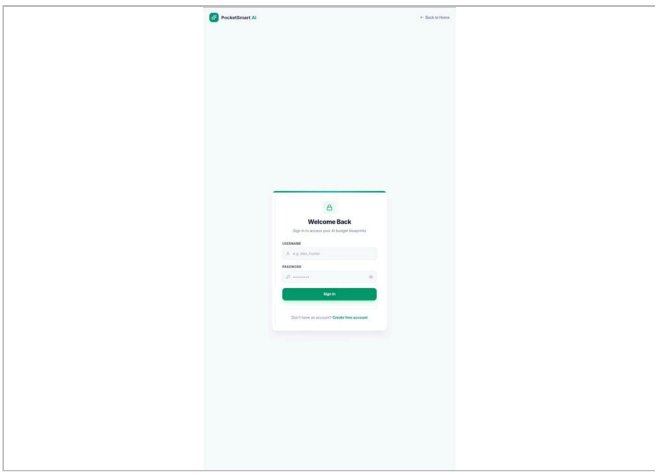
- base.html shows Login / Get Started for visitors and Dashboard, Home, Party, Jewelry and Logout for logged-in users.
- index.html introduces the three planners and how the app works; register.html and login.html hold the account forms.



Landing page (index.html)



Register page



Login page

Activity 4.2: Dashboard, planner forms and saved plans

- dashboard.html shows three planner cards and the latest 10 plans with date and budget.
- home.html, party.html and jewelry.html hold the planner forms and show the result panel; button text changes to Planning... while a request runs.

Milestone 5: Testing & Optimization

This milestone checks that the planners behave correctly and that the interface works on different screens. Plans were tested with real-world budgets for each domain; the screens below show the pages rendered from the project's own templates and style sheet with example data.

Activity 5.1: Test with real-world budget inputs

- Home: Rs 50,000 for a living room and bedroom with a sofa, fans, lights, TV unit and curtains.
- Party: Rs 80,000 birthday party for 60 guests in Salem with vegetarian food.
- Jewelry: Rs 40,000 for a wedding reception with an elegant gold style, with and without an outfit image.

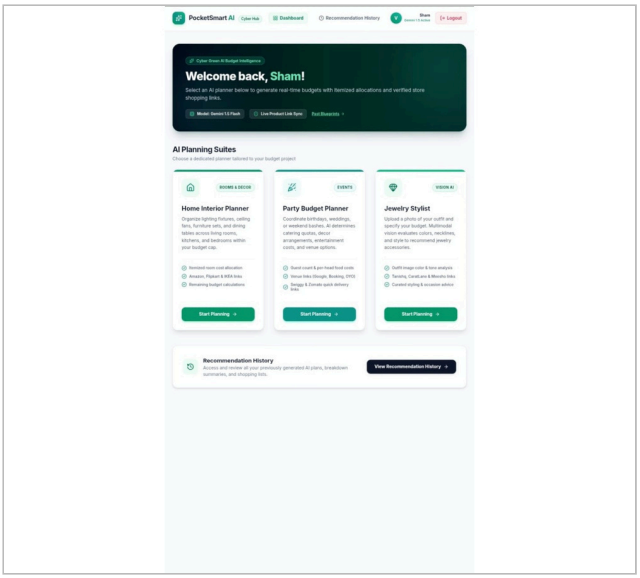
Activity 5.2: Budget adherence and prompt tuning

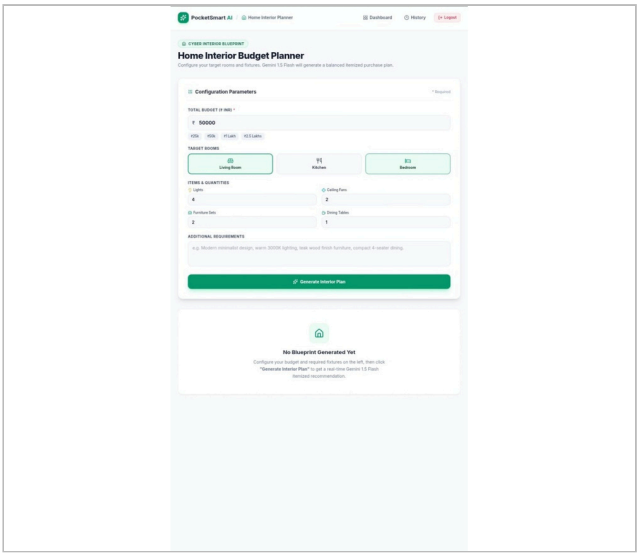
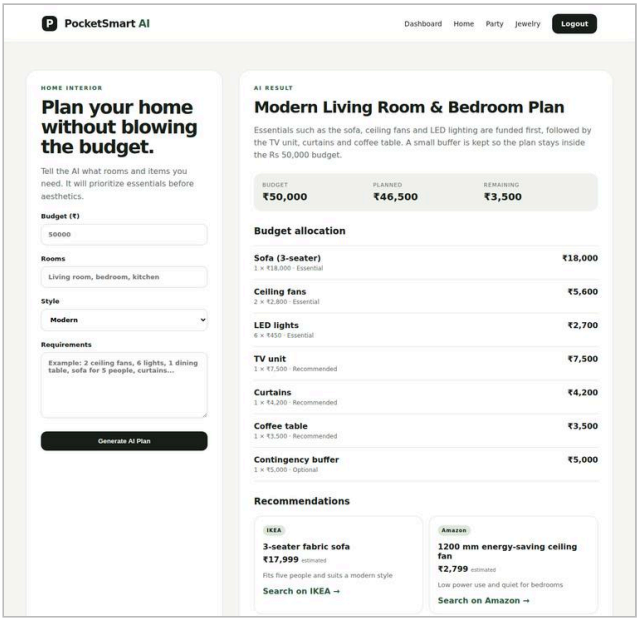
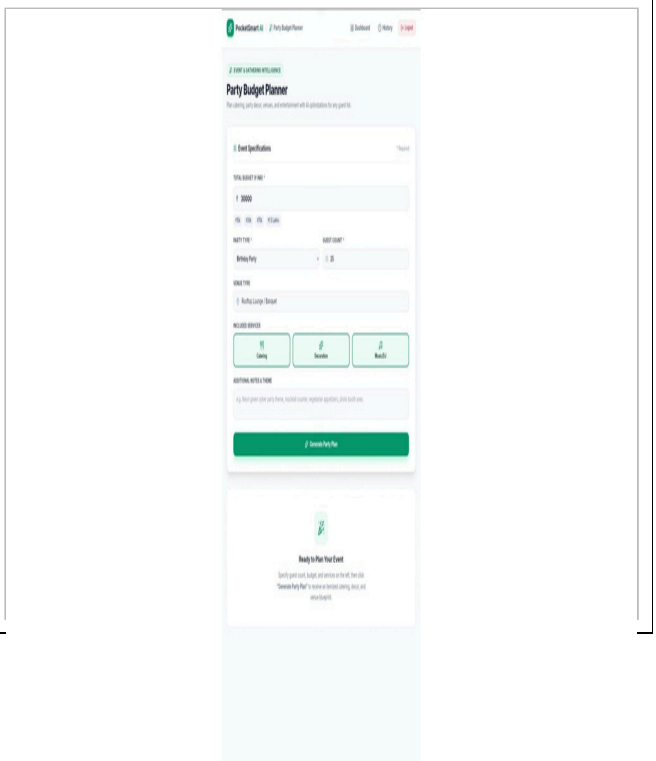
- The prompt requires allocated_total to equal the sum of the allocations and to stay within the budget.
- Validation rejects missing or non-positive budgets, guest counts below 1, missing required fields and unsupported image types.

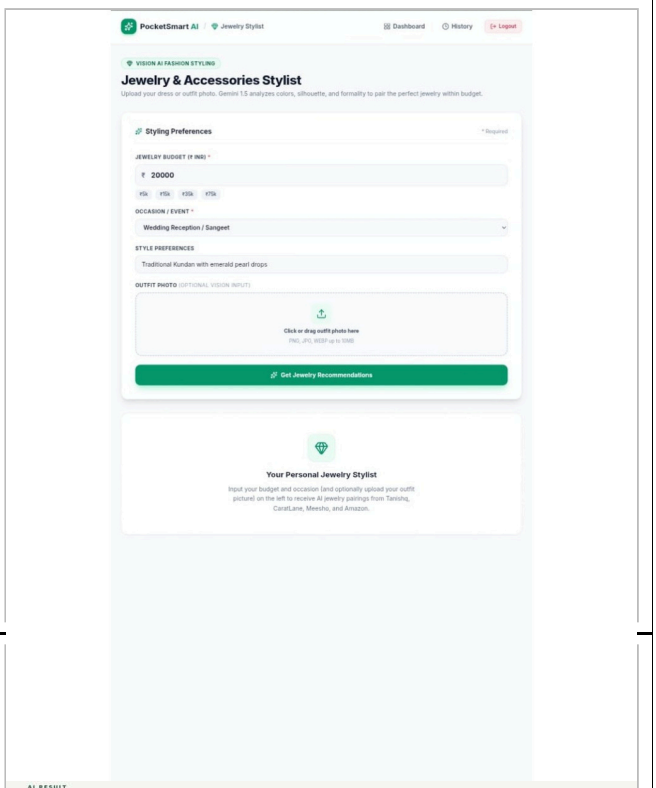
Activity 5.3: Load behaviour

Page loads and planner requests were load-tested with up to 25 concurrent users; the results are in the Performance Testing document.

Application Pages and Output Screens

Page	Description	Screenshot
Dashboard	Planner cards and the latest plans with date and budget.	

Page	Description	Screenshot
Home Interior Planner Page	Collects budget, rooms, style and requirements.	
Home Interior Result	Budget strip, allocations, recommendation cards and tips.	
Party Planner Result	Split across catering, venue, decoration, entertainment and contingency.	

Page	Description	Screenshot
Jewelry Planner Result	Jewelry options matched to occasion, style and budget.	
Saved Plan (History) Page	Re-opens a previously generated plan from the dashboard.	

Note: the screens show example plan data rendered through the project's templates; a live plan shows the Gemini response in the same layout.

Source code: https://github.com/santhoshcreater/pocket-smart-AI.git

Conclusion

PocketSmart AI helps people plan everyday spending on home interiors, parties and jewelry. By combining Flask, SQLite and Google Gemini, the application turns a budget, preferences and an optional outfit image into an allocation plan, recommendation cards and platform search links, and keeps every plan in the user's own history.

From the Gemini integration and the JSON prompt to the login system, the planner route and the responsive interface, the project was built to be modular and easy to run: one AI module, one planner route and shared templates serve all three planners. Future work includes live prices from official platform APIs, CSRF protection and rate limiting, PostgreSQL, and more planners.